

HASHDIT

HashDit

Smart Contract Security

ListaDao OEVLiquidator

Security Audit Report

ListaDao

Oct 27 2026 – Oct 29 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	Oct 27 2026 - Oct 29 2026
Overall Risk [Optional]	**Low**
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/258
Commit	feat/oev-liquidator

Smart Contract List

No.	Contract Name	Verdict	Details
1	OEVLiquidator.sol	See Findings	See Findings
2	deploy_oevLiquidator.sol	See Findings	See Findings
3	deploy_oevLiquidatorImpl.sol	See Findings	See Findings

Findings

[L01] The OEV role is global, so one deployment grants access to every whitelisted market	
Contract	OEVLiquidator.sol
Severity	Low
Description	**Description** The OEV role in the contract has no per market whitelist control, and a recent change removed the contract's own internal market whitelist. As a result, any address holding the OEV role can liquidate on every market where the OEVLiquidator contract is listed in Moolah's liquidation whitelist. **Impact** It is not possible to restrict one searcher to one market and a different searcher to another within a single deployment. If several markets are intended to be served by different searcher operators, they must use separate OEVLiquidator deployments. A single compromised or misbehaving OEV key therefore has a blast radius equal to every market that the shared deployment is authorized for.
Recommendation	If different markets are operated by different searchers, deploy a separate OEVLiquidator instance per operator and whitelist each instance only on its own markets i.e. only one OEV role per OEVLiquidator If not, if a single deployment must serve multiple operators, consider reintroducing per market gating inside the contract so that an OEV holder can be constrained to specific market
Status	Acknowledged, this is presumed to be contained only by the operational practice of deploying a separate OEVLiquidator per searcher/operator, which is a process commitment rather than an on-chain control.

[L02] OEV exclusivity depends on an unenforced Moolah whitelist configuration	
Contract	OEVLiquidator.sol
Severity	Low
Description	**Description** For a privileged searcher to hold the exclusive right to act on a freshly pushed price, each target market in Moolah must list only the OEVLiquidator contract in its liquidation whitelist. This condition is set through Moolah governance and is neither enforced nor checked by the OEVLiquidator. **Impact** If the market whitelist also contains other liquidator contracts or bots, or if it is empty which makes the market open to everyone, then ordinary liquidators compete on the same fresh price and the OEV leaks to whoever transacts first. In that case the OEV role restriction provides no economic exclusivity at all.

Recommendation	Depending on the required configuration of specific Moolah markets, utilize the exclusive whitelist configuration as a potential operational requirement or document the required configuration when required.
Status	Acknowledged, it is presumed that a market is truly exclusive to the OEV searcher remains a property of Moolah governance configuration that the OEVLiquidator neither sets nor checks, so this stands as an accepted operational requirement that should be documented.

[L03] Making a market OEV exclusive removes liquidation liveness with no automatic fallback	
Contract	OEVLiquidator.sol
Severity	Low
Description	**Description** For a specific market, restricting a market so that only the OEVLiquidator may liquidate it is possibly what makes OEV capture possible, but it also locks out every other liquidator. **Impact** If the single searcher goes offline, the Atlas infrastructure is unavailable, or the OEV key is revoked, no liquidations can occur for that market and bad debt can accumulate. Pausing the OEVLiquidator only makes the situation worse. Comparable designs release the fresh price to the public after a bounded delay so that ordinary liquidators can still act as a fallback. This design has no such automatic fallback, so recovery requires manual reconfiguration of the market whitelist by Moolah governance.
Recommendation	Define and document a recovery procedure that specifies who re opens a market to other liquidators, and under what conditions, if the searcher fails.
Status	Acknowledged, it is presumed that If a market is made exclusive and the searcher or Atlas infrastructure fails, liquidations will be performed outside of the OEVLiquidator with appropriate liquidation roles and mechanisms.

[I01] OEV value capture happens entirely off chain and is not enforced on chain	
Contract	OEVLiquidator.sol
Severity	Informational
Description	**Description** OEVLiquidator forwards the full amount of seized collateral to the searcher that calls it, which is the entire liquidation incentive that Moolah grants. The contract has no fee split, no bonus accounting, and no protocol share, unlike comparable OEV designs that retain a configurable portion of each liquidation's profit for the protocol. JavaScript

```
 function liquidate(
    bytes32 id,
    address borrower,
    uint256 seizedAssets,
    uint256 repaidShares
  ) external nonReentrant whenNotPaused onlyRole(OEV) returns (uint256,
uint256) {
    IMoolah.MarketParams memory params =
IMoolah(MOOLAH).idToMarketParams(id);
    (uint256 _seizedAssets, uint256 _repaidAssets) =
IMoolah(MOOLAH).liquidate(
      params,
      borrower,
      seizedAssets,
      repaidShares,
      abi.encode(params.loanToken, msg.sender)
    );
    params.collateralToken.safeTransfer(msg.sender, _seizedAssets);
    return (_seizedAssets, _repaidAssets);
  }
```

Impact
The design assumes the OEV value is extracted beforehand through an off chain Atlas oracle update push in which searchers bundle the price update with liquidation. If the OEV role is granted to a single searcher rather than being driven by a competitive auction/price update, there is no on chain competition and the protocol captures none of the OEV. Because the capture is off chain, it cannot be verified or guaranteed by the contract code.

Recommendation	Decide the capture model explicitly. Either document and contractually bind the Atlas auction that is expected to extract the value, or add an on chain fee recipient and a collateral split so that a defined portion of each liquidation profit is retained by the protocol. The split provides a fallback that does not depend on trusting off chain infrastructure.
Status	Acknowledged, it is presumed that all OEV design considerations will be implemented either on the Atlas searcher side.

[I02] OEVLiquidator does not emit a Liquidation event, preventing per searcher attribution	
Contract	OEVLiquidator.sol
Severity	Informational
Description	**Description** OEVLiquidator does not emit a Liquidated event when a liquidation completes. The sibling contracts all emit such an event and record the liquidator address in it. Moolah does emit its own Liquidate event, but that event records msg.sender as this contract rather than the searcher who initiated the liquidation.

JavaScript

```
 function liquidate(
    bytes32 id,
    address borrower,
    uint256 seizedAssets,
    uint256 repaidShares
  ) external nonReentrant whenNotPaused onlyRole(OEV) returns (uint256,
uint256) {
    IMoolah.MarketParams memory params =
IMoolah(MOOLAH).idToMarketParams(id);
    (uint256 _seizedAssets, uint256 _repaidAssets) =
IMoolah(MOOLAH).liquidate(
      params,
      borrower,
      seizedAssets,
      repaidShares,
      abi.encode(params.loanToken, msg.sender)
    );
    params.collateralToken.safeTransfer(msg.sender, _seizedAssets);
    return (_seizedAssets, _repaidAssets);
  }
```

Impact
As a result, there is no on chain record that ties a liquidation back to the specific searcher, making per searcher attribution impossible for off chain monitoring, accounting, or reward systems.

Recommendation	Consider emitting a dedicated Liquidated event from OEVLiquidator that includes the searcher address along with the relevant liquidation details, matching the pattern used by the sibling contracts.
Status	Fixed, now includes the event Liquidated(id, borrower, seizedAssets, repaidAssets, repaidShares, liquidator) with the same signature as PublicLiquidator, emitted at the end of liquidate() with msg.sender (the searcher) as liquidator, giving per-searcher attribution. Note that the event's repaidShares field carries the caller input, which is 0 on seize-specified liquidations (matching PublicLiquidator's convention). The actual repaid amount is in _repaidAssets, so off-chain consumers should key on repaidAssets, not repaidShares, for the realized value.

[I03] Missing test coverage for liquidate, access control, upgrade, and reentrancy paths	
Contract	OEVLiquidator.sol
Severity	Informational
Description	**Description**

	The OEVLiquidator test suite only covers a portion of the contract's behavior: the happy path liquidation, the OEV only access check on liquidate(), the pause and unpause flow, and the Moolah only guard on onMoolahLiquidate(). Several important paths are never tested. 1. First, the repaidShares branch of liquidate(), where seizedAssets is 0 and Moolah derives the seized amount, is never exercised. 2. Second, liquidation against a market whose Moolah liquidation whitelist is non-empty and names only this contract is not tested, and neither is the revert that occurs when this contract is not whitelisted. Because the test market uses an empty whitelist, it is open to everyone, so the core premise of the OEV design, that a market whitelists only this contract, is never verified. 3. Third, rotation of the OEV role by MANAGER through grantRole and revokeRole is untested. 4. Fourth, the MANAGER only access control on withdrawERC20() is untested. 5. Fifth, contract upgrades through the UUPS _authorizeUpgrade() gate are untested, including state preservation across an upgrade and the fact that the immutable MOOLAH value must be re-supplied correctly on each new implementation. 6. Sixth, reentrancy attempts through the callback or through a malicious token are untested. **Impact** There is no exploitable defect on any of these paths, but leaving them uncovered means that future regressions in access control, in the whitelist gating that underpins OEV exclusivity, in role rotation, and in upgrade safety could ship without detection.
Recommendation	Consider extending the test suite to cover the missing paths.
Status	Fixed, the test suite was extended to cover all six previously-uncovered paths.

[I04] Deploy template .env.example omits required variables and misnames the admin entry	
Contract	deploy_oevLiquidator.sol
Severity	Informational
Description	**Description** The deployment scripts read several environment variables that the .env.example template does not document. The full deployment script reads MOOLAH, TIMELOCK, MANAGER, PAUSER, and OEV. The implementation only script reads MOOLAH. The shared deploy base reads PRIVATE_KEY on BSC mainnet and PRIVATE_KEY_TESTNET on test networks. The template documents PRIVATE_KEY and PAUSER, but it omits PRIVATE_KEY_TESTNET, MOOLAH, TIMELOCK, MANAGER, and OEV. It also contains an entry named ADMIN, yet no script reads ADMIN. The address that becomes DEFAULT_ADMIN_ROLE is taken from TIMELOCK instead.

	Impact A deployer who relies on the template will be missing the required variables, so the scripts revert when those variables are read. That failure is safe and does not deploy a broken contract. The real problem is operational clarity. The ADMIN versus TIMELOCK naming mismatch can mislead a deployer about which variable controls the admin role, and the missing entries give no guidance on what must be configured before deploying.
Recommendation	Consider updating .env.example to list every variable the scripts actually read, using the exact names the scripts expect
Status	Acknowledged, it is presumed that necessary adjustments to the env file will be performed during deployment of the OEVLiquidator.

[I05] Missing profitability check in OEVLiquidator leaves searchers unprotected	
Contract	OEVLiquidator.sol
Severity	Informational
Description	**Description** Every sibling contract, including PublicLiquidator, Liquidator, and V3Liquidator, snapshots balances before and after the liquidation and reverts when the operation is not profitable. OEVLiquidator performs no such check. Note that the searcher already holds the seized collateral and knows repaidAssets inside onOEVLiquidate, so it can possibly compute net profit and revert if unprofitable. **Impact** While this is less critical here than in the sibling contracts, the absence of a profitability or output bound means a searcher has no on chain guarantee that the liquidation leaves them in a non losing position, so an unprofitable execution can settle without reverting.
Recommendation	Consider adding a profitability guard consistent with the sibling liquidator contracts, if not, document that profitability relies on the OEV onOEVLiquidate callback itself
Status	Acknowledged, it is presumed that a profitability check will and should be included within the Atlas onOEVLiquidate callback.

[I06] Residual loan token in OEVLiquidator can be consumed by another liquidator instead of being recovered	
Contract	OEVLiquidator.sol
Severity	Informational
Description	**Description** In the push based callback flow, OEVLiquidator never verifies that the loan token repayment originates from the payer's onOEVLiquidate callback. Inside onMoolahLiquidate, the contract transfers the seized collateral to the payer, calls payer.onOEVLiquidate(repaidAssets, payload), and then approves Moolah for exactly repaidAssets. Moolah afterwards pulls repaidAssets from OEVLiquidator's own token

balance via transferFrom, with no check that the balance increased by repaidAssets during the callback.

```javascript
 function onMoolahLiquidate(uint256 repaidAssets, bytes calldata data)
external {
    require(msg.sender == MOOLAH, OnlyMoolah());
    (
      address loanToken,
      address collateralToken,
      uint256 collateralTokenPreBalance,
      address payer,
      bytes memory payload
    ) = abi.decode(data, (address, address, uint256, address, bytes));
    // transfer seized collateral to payer first
    collateralToken.safeTransfer(payer,
collateralToken.balanceOf(address(this)) - collateralTokenPreBalance);
    // callback payer
    IOEVLiquidateCallback(payer).onOEVLiquidate(repaidAssets, payload);
    // Note: should be reverted if payer doesn't transfer sufficient
loan token here
    loanToken.safeApprove(MOOLAH, repaidAssets);
  }
```

The inline comment stating that the call reverts when the payer does not transfer enough loan token only holds when the contract's loan token balance started at zero. The design tolerates residual balances, because Moolah pulls only the exact approved amount, so over sending leaves a remainder.

Impact
If a residual or stuck loan token balance >= repaidAssets is present, a searcher can transfer nothing, or less than repaidAssets, inside onOEVLiquidate and still receive the full seized collateral, with the repayment covered by the pre existing funds. The core problem is that excess or stuck loan token left on the contract becomes available to be utilized by another potential liquidator to fund their own repayment, rather than being reserved for recovery. Those residual funds may belong to third parties, to the protocol, or to another searcher's over send, so one liquidator can effectively spend another party's funds.

Recommendation	Consider validating that the repayment is funded by the current liquidation rather than by a pre existing balance. Snapshot the loan token balance after the seized collateral is transferred out and immediately before invoking onOEVLiquidate, then after the callback require that balanceAfter - balanceBefore >= repaidAssets. This prevents excess funds from being used and allows the MANAGER role to pull any excess funds.
Status	Acknowledged, it is presumed that it is by design that any excess or donated loan tokens residing in the OEVLiquidator can either be retrieved back by the manager or used to subsequent liquidations, whichever actions takes place first.

[I07] OEVLiquidator market configuration considerations

Contract	OEVLiquidator.sol
Severity	Informational
Description	**Description** OEVLiquidator forwards directly to Moolah.liquidate without any awareness of, or constraint on, the oracle configuration of the market being liquidated. It relies entirely on Moolah's per-market liquidationWhitelist to decide which markets it may act on.. The OEV searcher atomically publishes the Atlas oracle price in the same bundle and then calls OEVLiquidator.liquidate, so whether the OEV capture is safe depends entirely on which slot Atlas occupies in each market's multi-oracle configuration, and OEVLiquidator enforces nothing about this. **Impact** Four concrete conditions follow. 1. First, for markets where Atlas is the pivot and the main equals the fallback (USDT, FDUSD, ETH, and USDF, which are common loan tokens), drift between Atlas and ChainLink, or a deliberate in bundle Atlas publish, pushes the ChainLink main out of band. The identical ChainLink fallback is then re validated against the same pivot, also fails, and the oracle reverts. This makes peek(loanToken) revert, so Moolah.liquidate and OEVLiquidator.liquidate revert, and the position cannot be liquidated even when its collateral feed is perfectly healthy, exposing the protocol to bad debt in any market that uses one of these assets as the loan token. 2. Second, for the bStock markets where Atlas is the main, APRO is the pivot, the fallback is also Atlas, and the band is 1.02 / 0.98 (plus or minus 2%), any move greater than 2%, which is routine for tokenized equities, pushes the Atlas main out of band, the Atlas fallback fails the same bound, and the oracle reverts, so liquidation is blocked exactly when it is most needed and bad debt accrues. While inside the plus or minus 2% band around a drifted APRO, the searcher publishes at the profitable edge of the band, liquidating up to 2% away from fair value and over seizing collateral from the borrower, which OEVLiquidator forwards straight to the searcher. 3. Third, for the roughly 110 assets where Atlas is the main with no pivot and no fallback, including LISTA and the long tail tokenized equities, the liquidation price is exactly what the searcher published, with no sanity bound to catch a stale, manipulated, or buggy value. Because the OEV role is the same entity that publishes the Atlas price, calls liquidate, and receives the seized collateral, one party controls the price, the action, and the proceeds end to end, and stale, weekend, or halted equity feeds let liquidations execute at prices disconnected from reality. 4. Fourth, for markets where Atlas is the pivot with a distinct fallback, where BTCB and solvBTC fall back to RedStone and BNB has a RedStone OEV main with a ChainLink fallback, the searcher's in bundle Atlas publish, acting as the pivot, can invalidate the main and force the price onto the fallback, letting the searcher choose whether a position is liquidatable and then liquidate it through OEVLiquidator at the more favorable source. The common root cause is that OEVLiquidator grants an OEV liquidating the oracle topology of the market, and

	OEV capture is only coherent when the searcher's publish actually sets the price, which is only true when Atlas is the main feed.
Recommendation	Consider restricting the markets OEVLiquidator may liquidate to a safe oracle configuration, enforced on chain in OEVLiquidator through a MANAGER maintained allowlist checked at the start of liquidate(), rather than relying only on Moolah's external liquidationWhitelist.
Status	Acknowledged, it is presumed that the above configurations will be considered when allowing a specific OEVLiquidator to perform liquidations on Moolah markets.